

SE3082 – Parallel Computing

Exercise 2 Details, Code & Output Verification



IT24103470
Kathisan A

Exercise 2 Details, Code & Output Verification

1. Exercise Task Description

Run the provided code in the AWS EC2 instance / Linux instance and briefly describe how the `#pragma omp single` command works in this program. **2. Complete C Source**

Code (exercise2.c)

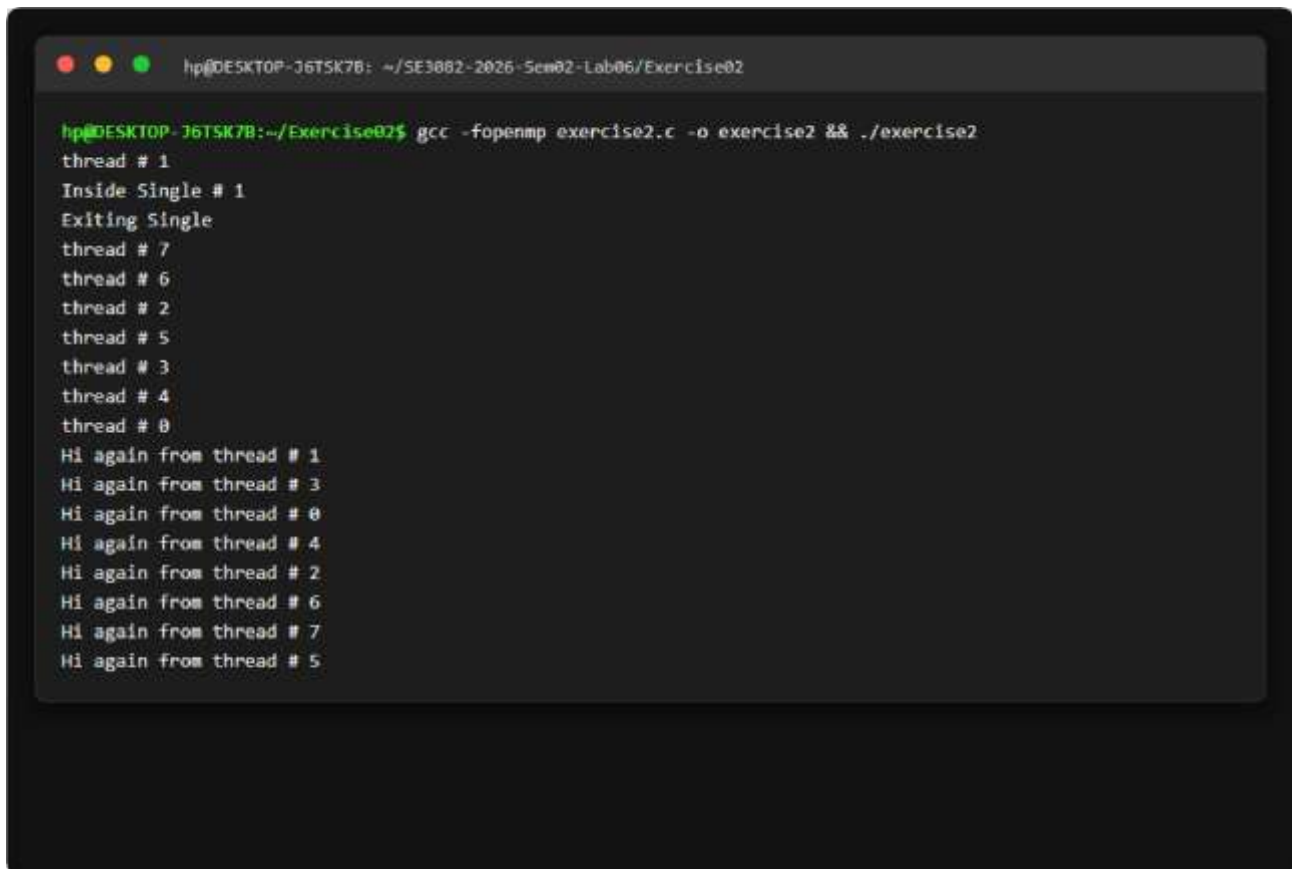
```
#include <omp.h>
#include <stdio.h>

int main() {
#pragma omp parallel
{
    int th = omp_get_thread_num();
    printf("thread # %d\n", th);

    #pragma omp single
    {
        printf("Inside Single # %d\n", omp_get_thread_num());
        printf("Exiting Single \n");
    }

    printf("Hi again from thread # %d\n", omp_get_thread_num());
}
    return 0;
}
```

3. Terminal Execution Output Screenshot

A terminal window with a dark background and light green text. The window title is 'hp@DESKTOP-36TSK7B: ~/SE3882-2026-Sem02-Lab06/Exercise02'. The prompt is 'hp@DESKTOP-36TSK7B:~/Exercise02\$'. The command entered is 'gcc -fopenmp exercise2.c -o exercise2 && ./exercise2'. The output shows thread execution order and a single-threaded block.

```
hp@DESKTOP-36TSK7B:~/Exercise02$ gcc -fopenmp exercise2.c -o exercise2 && ./exercise2
thread # 1
Inside Single # 1
Exiting Single
thread # 7
thread # 6
thread # 2
thread # 5
thread # 3
thread # 4
thread # 0
Hi again from thread # 1
Hi again from thread # 3
Hi again from thread # 0
Hi again from thread # 4
Hi again from thread # 2
Hi again from thread # 6
Hi again from thread # 7
Hi again from thread # 5
```

4. Directive Analysis & Explanation

How `#pragma omp single` works:

The `#pragma omp single` directive specifies that the enclosed structured block is executed by **only one thread** in the parallel team (not necessarily thread 0). All other threads wait at an *implicit barrier* at the end of the single construct before proceeding together.